

6.4

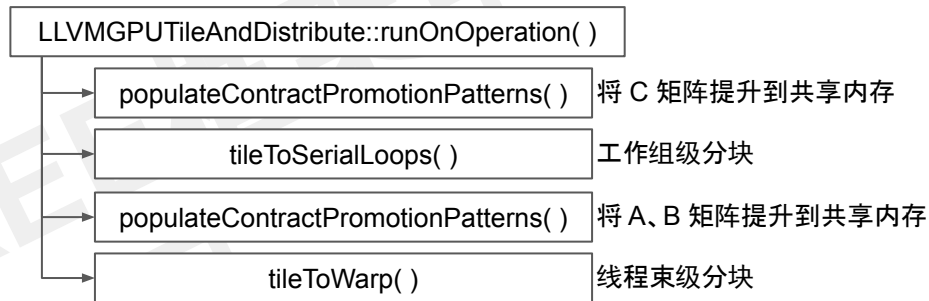
LLVMGPUTileAndDistribute pass 工作流程

输入 IR 分析

```
%cst = arith.constant 0.000000e+00 : f16
%c0 = arith.constant 0 : index
%0 = hal.interface.binding.subspan ... : !flow.dispatch.tensor<readonly:tensor<512x512xf16>>
%1 = hal.interface.binding.subspan ... : !flow.dispatch.tensor<readonly:tensor<512x512xf16>>
%2 = hal.interface.binding.subspan ... : !flow.dispatch.tensor<writeonly:tensor<512x512xf16>>
%3 = affine.apply affine_map<() [s0] -> (s0 * 32)>() [%workgroup_id_y]
%workgroup_id_x = hal.interface.workgroup.id[0] : index
%4 = affine.apply affine_map<() [s0] -> (s0 * 32)>() [%workgroup_id_x]
%subview = memref.subview %2[%3, %4] [32, 32] [1, 1] : memref<512x512xf16, #...<storage_buffer>> to
memref<32x32xf16, strided<[512, 1], offset: ?>, #...<storage_buffer>>
%subview_0 = memref.subview %0[%3, 0] [32, 512] [1, 1] : memref<512x512xf16, #...<storage_buffer>> to
memref<32x512xf16, strided<[512, 1], offset: ?>, #...<storage_buffer>>
%subview_1 = memref.subview %1[0, %4] [512, 32] [1, 1] : memref<512x512xf16, #...<storage_buffer>> to
memref<512x32xf16, strided<[512, 1], offset: ?>, #...<storage_buffer>>
linalg.fill {lowering_config = #iree_codegen.lowering_config<tile_sizes = [[32, 32, 32]]>} ins(%cst : f16) outs(%subview :
memref<32x32xf16, strided<[512, 1], offset: ?>, #...<storage_buffer>>)
linalg.matmul {lowering_config = #iree_codegen.lowering_config<tile_sizes = [[32, 32, 32]]>} ins(%subview_0, %subview_1 :
memref<32x512xf16, strided<[512, 1], offset: ?>, #...<storage_buffer>>, memref<512x32xf16, strided<[512, 1], offset: ?>,
#...<storage_buffer>>) outs(%subview : memref<32x32xf16, strided<[512, 1], offset: ?>, #...<storage_buffer>>)
return
```

LLVMGPUPTileAndDistribute pass 主要函数调用流程

1. 操作数内存提升(两次)
2. 工作组级分块过程实现
3. 线程束级分块过程实现



操作数的内存提升

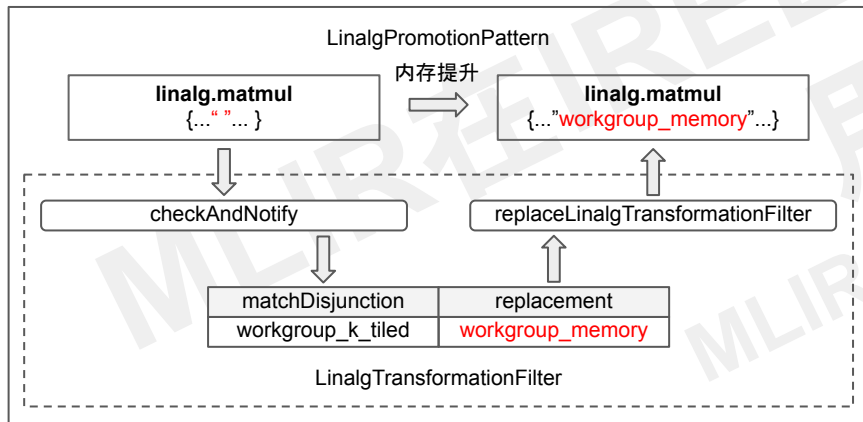


```
void populateContractPromotionPatterns(RewritePatternSet &patterns,
                                       ArrayRef<int64_t> operandsToPromote) {
    MLIRContext *context = patterns.getContext();
    patterns.insert<LinalgPromotionPattern<linalg::MatmulOp>,
                  LinalgPromotionPattern<linalg::BatchMatmulOp>,
                  LinalgPromotionPattern<linalg::GenericOp>>>(
        context,
        linalg::LinalgPromotionOptions()
            .setAllocationDeallocationFns(allocateWorkgroupMemory, memref.alloc
                                           deallocateWorkgroupMemory)
            .setCopyInOutFns(copyToWorkgroupMemory, copyToWorkgroupMemory) memref.copy
            .setOperandsToPromote(operandsToPromote)
            .setUseFullTileBuffers({false, false}),
        LinalgTransformationFilter(
            {StringAttr::get(context, getWorkgroupKTiledMarker())}, matchDisjunction
            StringAttr::get(context, getWorkgroupMemoryMarker()) replacement
            .setMatchByDefault()
            .addFilter(contractOpFilter));
}
```

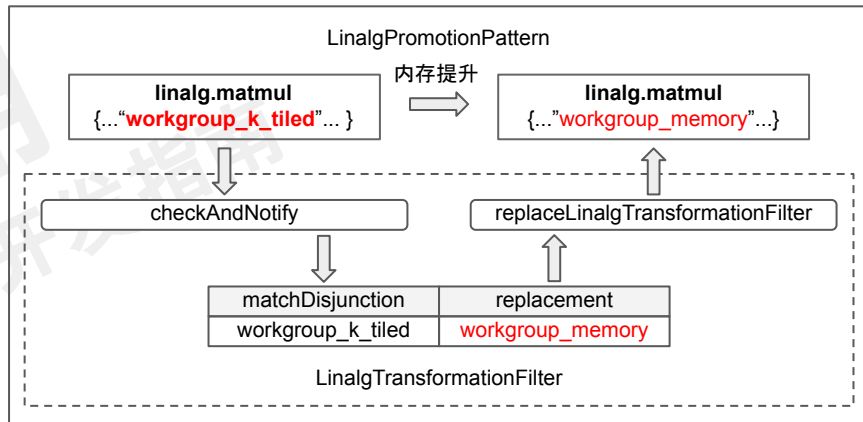
操作数的内存提升



C 操作数内存提升

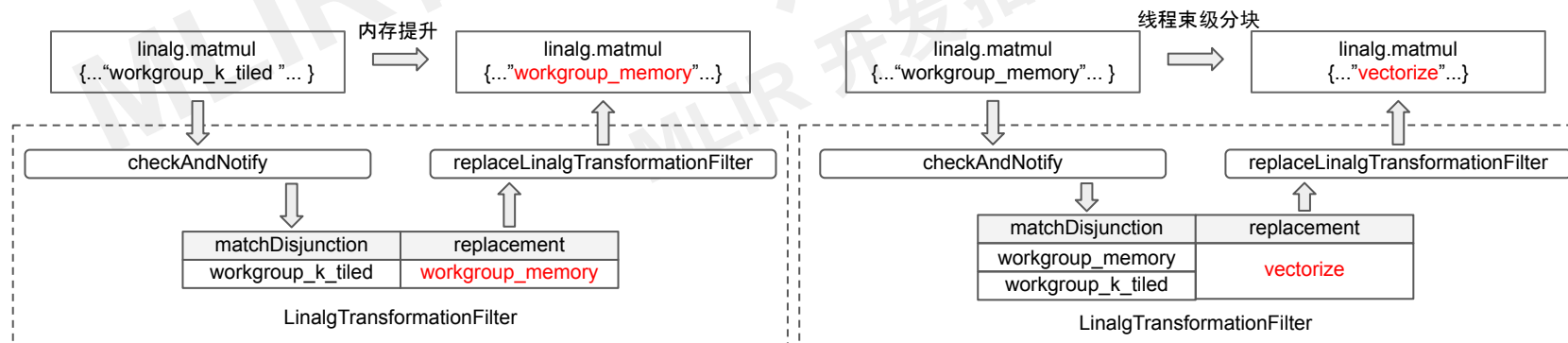
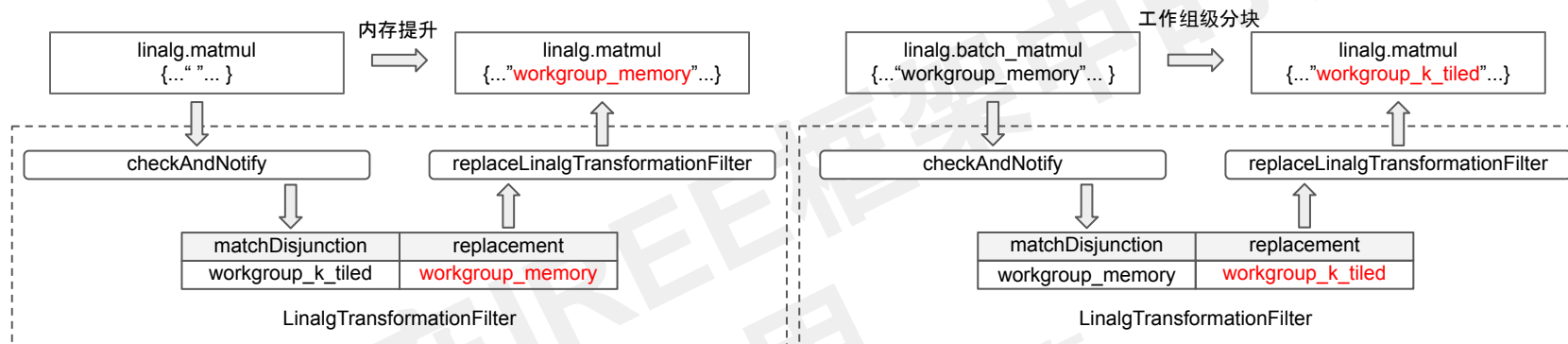


A/B 操作数内存提升

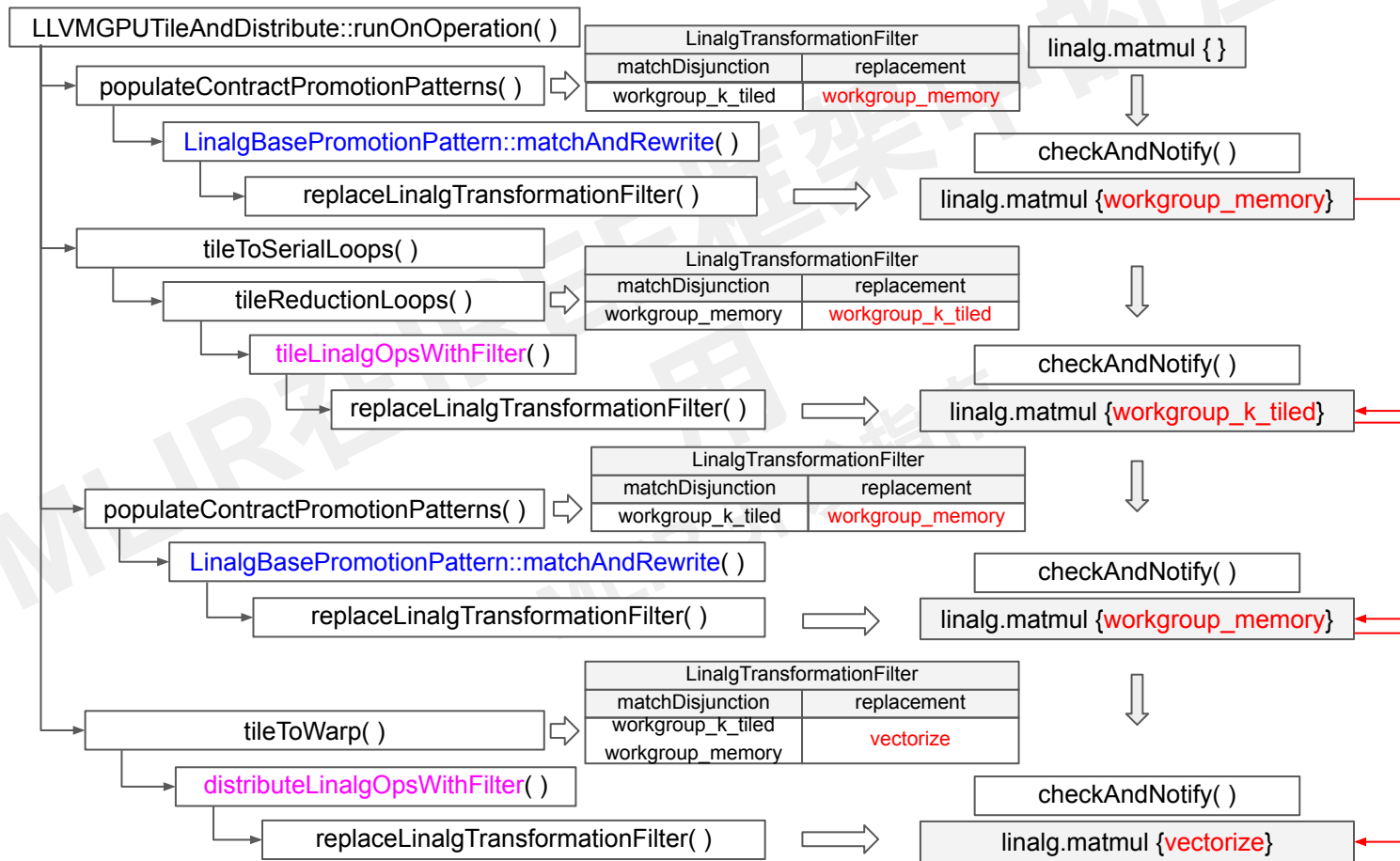


kLinalgTransformMarker: " ", "workgroup_memory", "workgroup_k_tiled", "vectorize".

kLinalgTransformMarker 属性值变化总结



驱动 kLinalgTransformMarker 属性值变化的函数



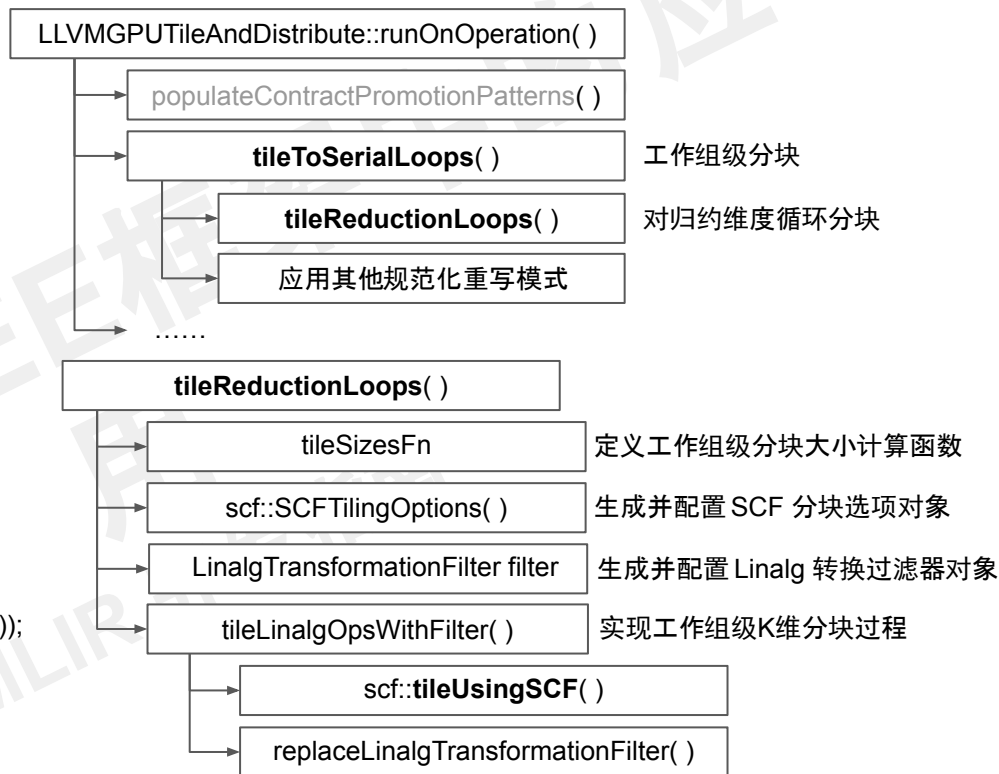
工作组级分块过程实现

tileReductionLoops()函数包含4部分功能：

- (1) 定义工作组级分块大小计算函数
- (2) 配置 SCF 分块选项对象
- (3) 配置 Linalg 转换过滤器对象
- (4) 实现工作组级 K 维分块过程

```
auto tileSizesFn = [](OpBuilder &builder,
    Operation *op) -> SmallVector<OpFoldResult> {
    auto interfaceOp = cast<PartitionableLoopsInterface>(*op);
    auto partitionedLoops =
        interfaceOp.getPartitionableLoops(kNumMaxParallelDims);
    SmallVector<OpFoldResult> tileSizes =
        getAsIndexOpFoldResult(op->getContext(), getTileSizes(op, 0));
    auto zeroAttr = builder.getIndexAttr(0);
    for (unsigned depth : partitionedLoops) {
        if (depth < tileSizes.size()) {
            tileSizes[depth] = zeroAttr;
        }
    }
    ...
};

auto tilingOptions =
    scf::SCFTilingOptions().setTileSizeComputationFunction(tileSizesFn);
```

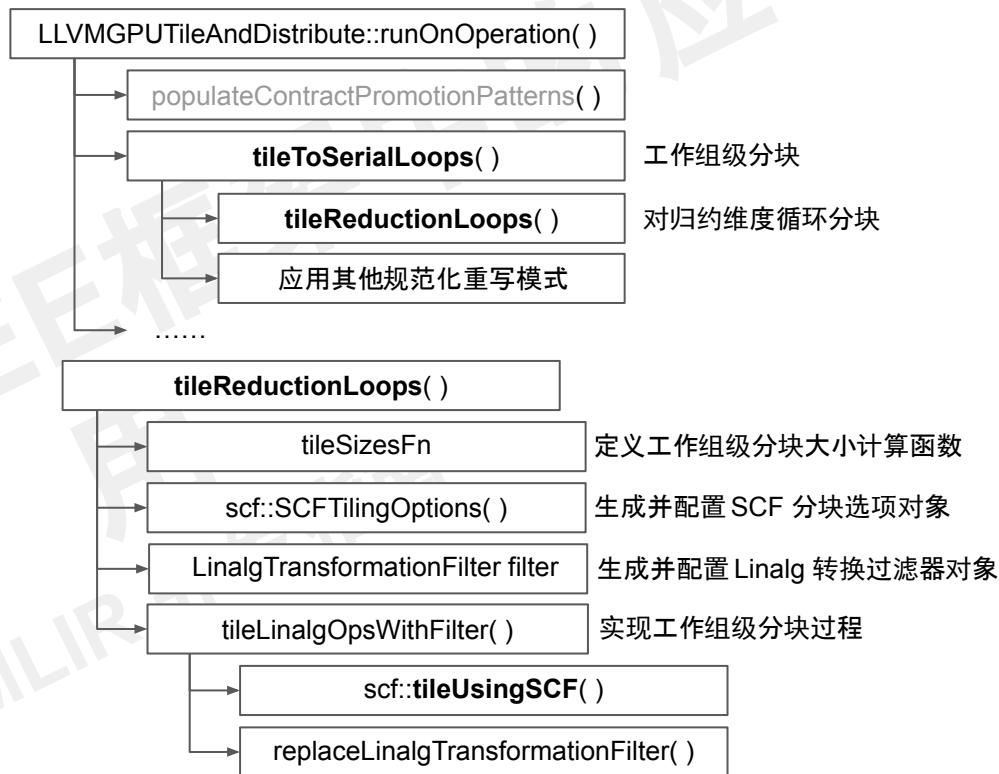


工作组级分块过程实现

tileReductionLoops()函数包含4部分功能：

- (1) 定义工作组级分块大小计算函数
- (2) 配置SCF 分块选项对象
- (3) Linalg 转换过滤器对象
- (4) 实现工作组级K维分块过程

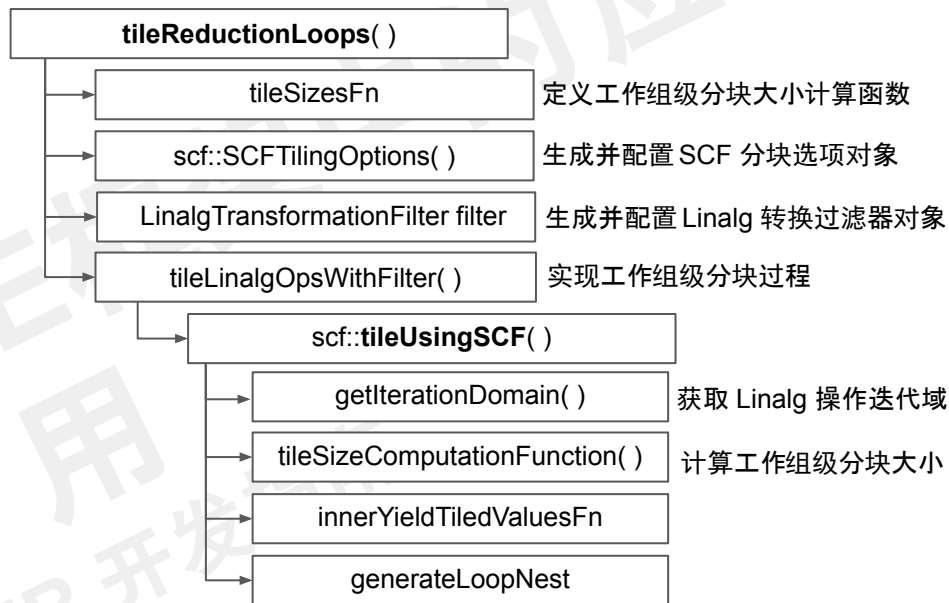
```
LinalgTransformationFilter filter(  
  ArrayRef<StringAttr>{  
    StringAttr::get(context, getWorkgroupMemoryMarker()),  
    StringAttr::get(context, getWorkgroupKTiledMarker());  
  },  
  matchDisjunction  
  replacement  
);
```



工作组级分块过程实现

scf::tileUsingSCF() 函数包含5部分功能：

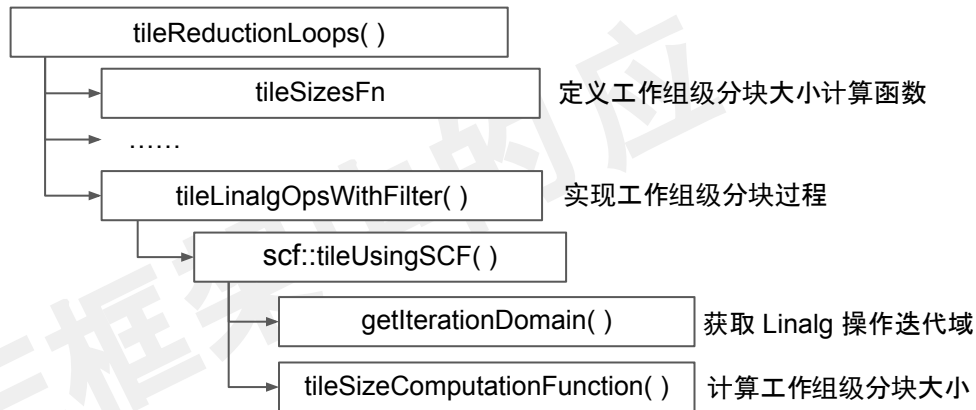
- (1) 获取 Linalg 操作迭代域
- (2) 计算工作组级分块大小
- (3) 定义工作组级分块参数计算函数
- (4) 生成工作组级分块循环体逻辑
- (5) 生成工作组级分块循环嵌套



工作组级分块过程实现

(1) 获取 Linalg 操作迭代域

(2) 计算工作组级分块大小



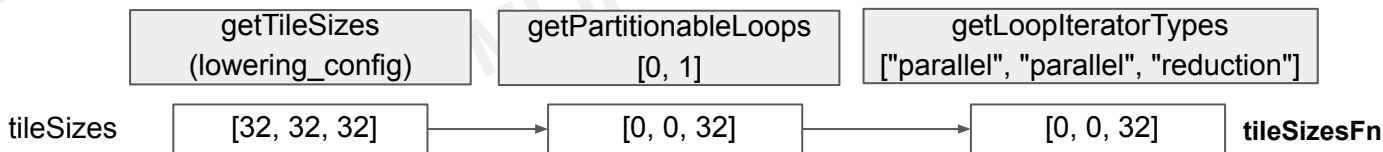
```
linalg.matmul {lowering_config = #iree_codegen.lowering_config<tile_sizes = [[32, 32, 32]]>} ins(%subview_0, %subview_1 :  
memref<32x512xf16, ...>, ..., memref<512x32xf16, ...>, ...) outs(%subview : memref<32x32xf16, ...>, ...)
```

```
struct Range {  
  OpFoldResult offset;  
  OpFoldResult size;  
  OpFoldResult stride;  
};
```

M N K

迭代域向量: [[0, 32, 1], [0, 32, 1], [0, 512, 1]]

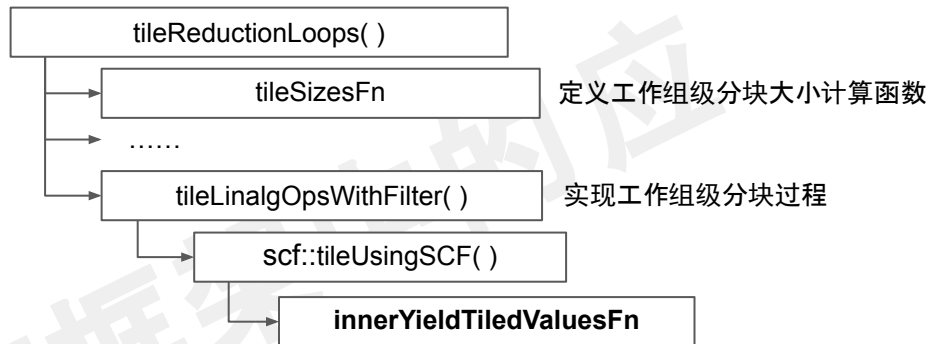
getIterationDomain()



linalg.matmul 操作的工作组级分块大小

工作组级分块过程实现

(3) 定义工作组级分块参数计算函数



```
auto innerYieldTiledValuesFn =
  [&](..., ValueRange ivs,...) -> LogicalResult {
    SmallVector<OpFoldResult> offsets, sizes;
    {
      int materializedLoopNum = 0;
      for (auto [tileSize, loopRange] :
        llvm::zip_equal(tileSizesVector, iterationDomain)) {
        if (isConstantIntValue(tileSize, 0)) {
          offsets.push_back(loopRange.offset);
          sizes.push_back(loopRange.size);
          continue;
        }
        Value iv = ivs[materializedLoopNum++];
        offsets.push_back(iv);
        sizes.push_back(
          getBoundedTileSize(rewriter, loc, loopRange, iv, tileSize));
      }
    }
    ...
  }
```

tileSizes = [0, 0, 32]

iterationDomain = [[0, 32, 1], [0, 32, 1], [0, 512, 1]]

ivs = [%arg0]

offsets = [0, 0, %arg0]

sizes = [32, 32, 32]

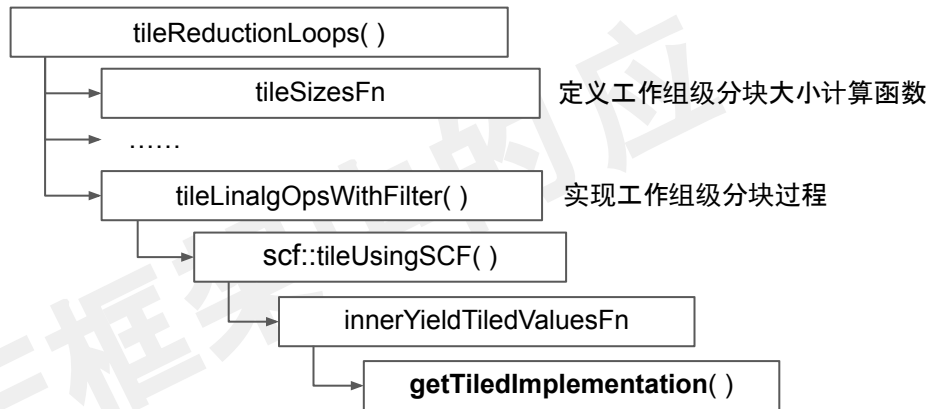
generateLoopNest() 函数生成分块循环嵌套结构:

```
scf.for %arg0 = %c0_6 to %c512 step %c32_7 {
  ^bb0(%arg0: index):
} : (index, index, index) -> ()
```

工作组级分块过程实现

(4) 生成工作组级分块循环体逻辑

```
offsets = [0, 0, %arg0]
sizes = [32, 32, 32]
```



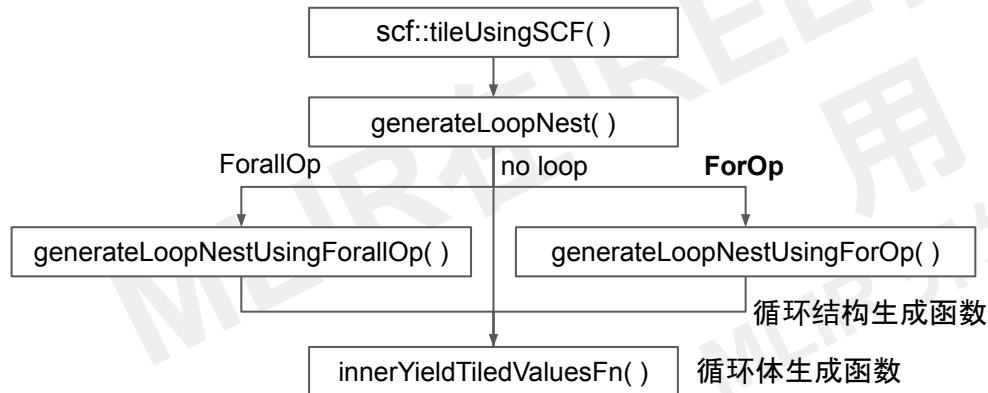
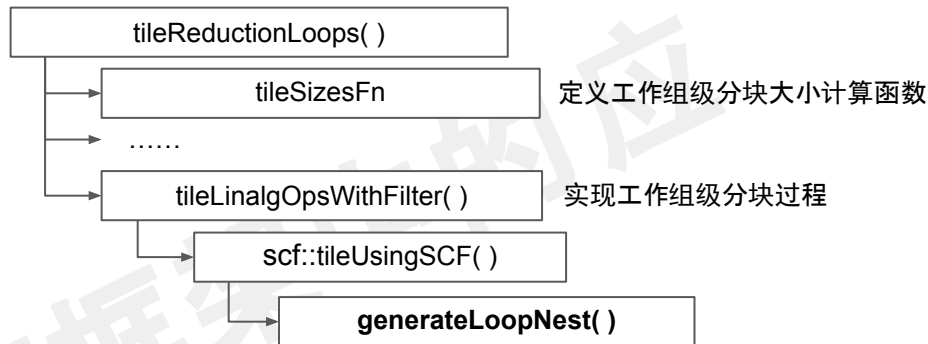
```
linalg.matmul {lowering_config = #iree_codegen.lowering_config<tile_sizes = [[32, 32, 32]]>} ins(%subview_0, %subview_1 :
memref<32x512xf16, ...>, ..., memref<512x32xf16, ...>, ...) outs(%subview : memref<32x32xf16, ...>, ...)
```

```
makeTiledShapes()
```

```
%subview_8 = memref.subview %subview_0[0, %arg0] [32, 32] [1, 1] : memref<32x512xf16, ...>, ...<storage_buffer>> to
memref<32x32xf16, ...>, ...<storage_buffer>>
%subview_9 = memref.subview %subview_1[%arg0, 0] [32, 32] [1, 1] : memref<512x32xf16, ...>, ...<storage_buffer>> to
memref<32x32xf16, ...>, ...<storage_buffer>>
%subview_10 = memref.subview %subview_2[0, 0] [32, 32] [1, 1] : memref<?x?xf16, strided<[32, 1]>, ...<workgroup>> to
memref<32x32xf16, strided<[32, 1]>, ...<workgroup>>
linalg.matmul {...} ins(%subview_8, %subview_9 : memref<32x32xf16, ...>, ...<storage_buffer>>,
memref<32x32xf16, ...>, ...<storage_buffer>>)
outs(%subview_10 : memref<32x32xf16, ...>, ...<workgroup>>)
```

工作组级分块过程实现

(5) 生成工作组级分块循环嵌套



tileSizes = [0, 0, 32]

iterationDomain = [[0, 32, 1], [0, 32, 1], [0, 512, 1]]

generateLoopNest() 函数生成分块循环嵌套结构:

```
scf.for %arg0 = %c0_6 to %c512 step %c32_7 {  
  ^bb0(%arg0: index):  
} : (index, index, index) -> ()
```

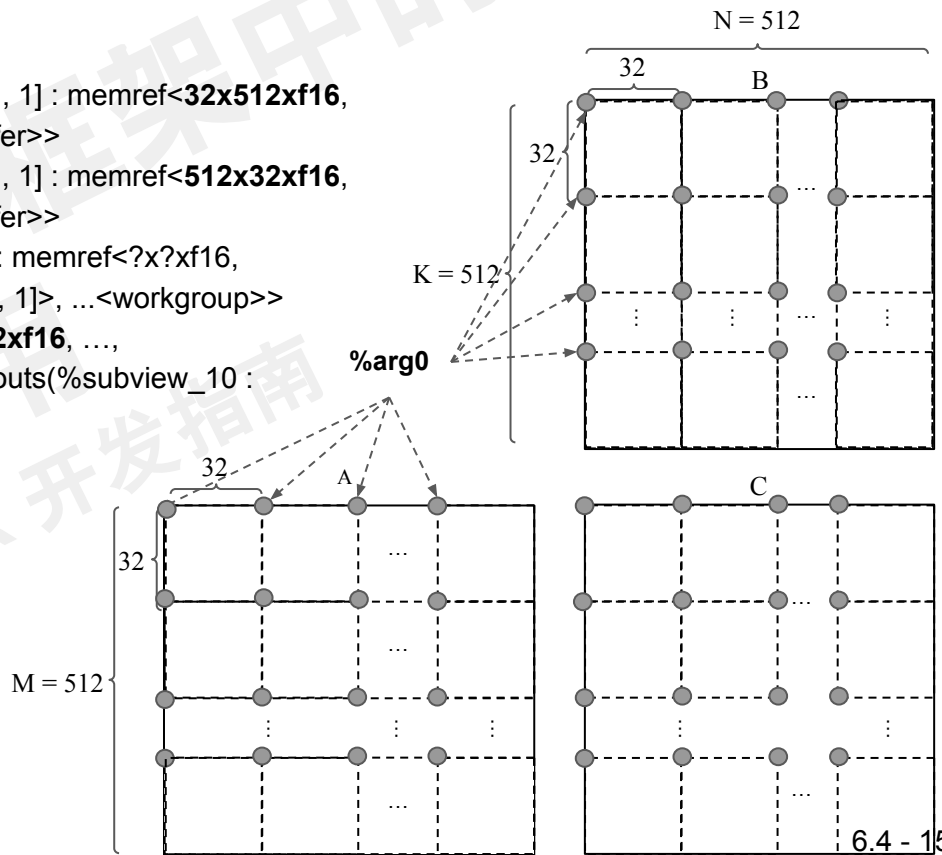
ivs = [%arg0]

工作组级分块过程实现

```

scf.for %arg0 = %c0_6 to %c512 step %c32_7 {
  %subview_8 = memref.subview %subview_0[0, %arg0] [32, 32] [1, 1] : memref<32x512xf16,
  ..., ...<storage_buffer>> to memref<32x32xf16, ..., ...<storage_buffer>>
  %subview_9 = memref.subview %subview_1[%arg0, 0] [32, 32] [1, 1] : memref<512x32xf16,
  ..., ...<storage_buffer>> to memref<32x32xf16, ..., ...<storage_buffer>>
  %subview_10 = memref.subview %subview_2[0, 0] [32, 32] [1, 1] : memref<?x?xf16,
  strided<[32, 1]>, ...<workgroup>> to memref<32x32xf16, strided<[32, 1]>, ...<workgroup>>
  linalg.matmul {...} ins(%subview_8, %subview_9 : memref<32x32xf16, ...,
  ...<storage_buffer>>, memref<32x32xf16, ..., ...<storage_buffer>>) outs(%subview_10 :
  memref<32x32xf16, ..., ...<workgroup>>)
}
...

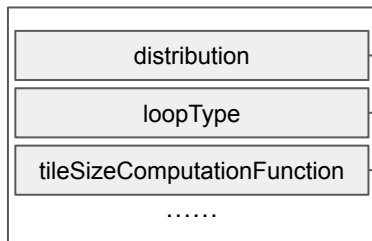
```



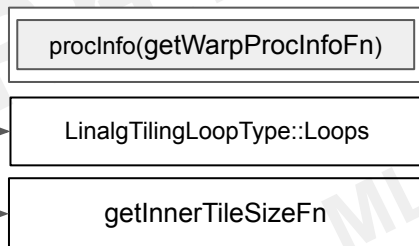
线程束级分块过程实现

- (1) 定义线程束级分块大小计算函数
- (2) 定义处理器信息函数
- (3) 配置 Linalg 循环分布选项对象
- (4) 配置 Linalg 分块选项对象
- (5) 配置 Linalg 转换过滤器对象

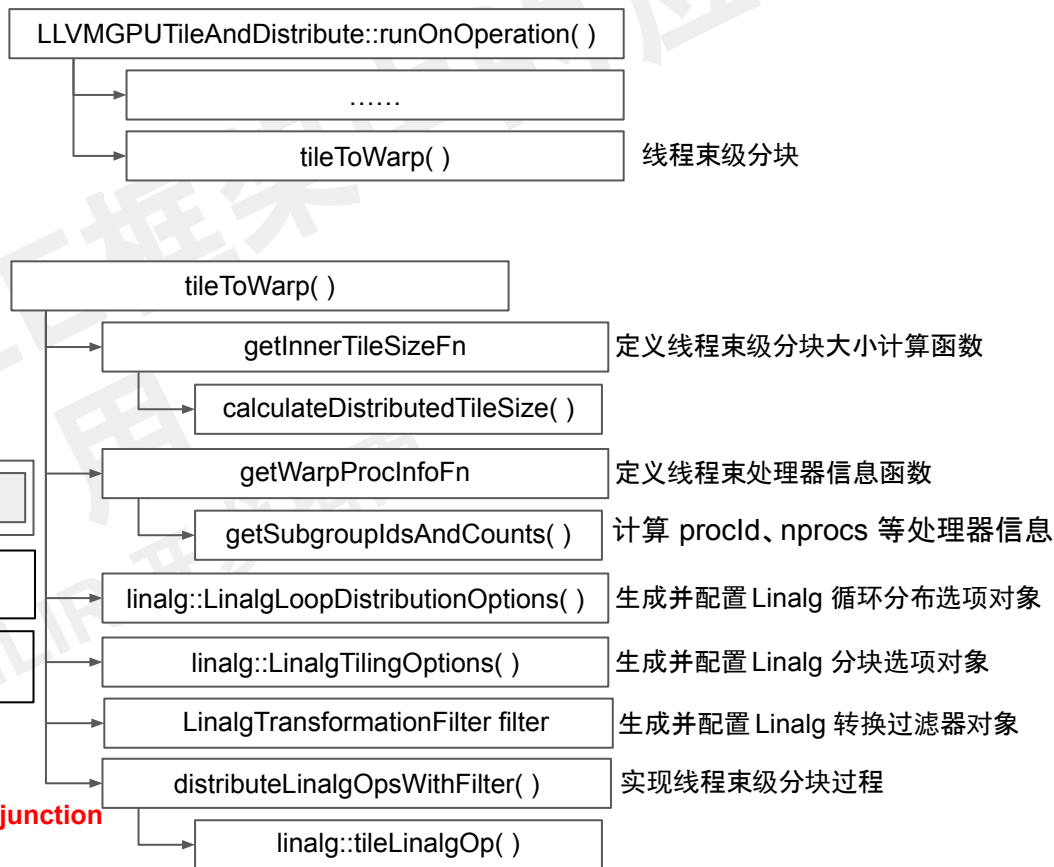
LinalgTilingOptions



LinalgLoopDistributionOptions

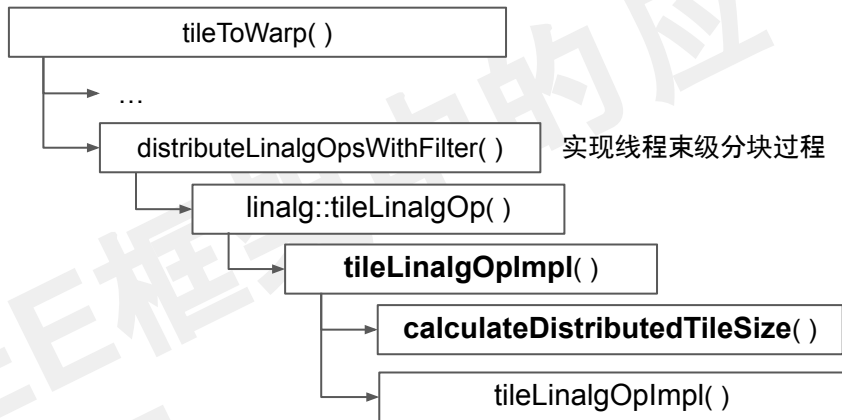


```
LinalgTransformationFilter filter(  
  {StringAttr::get(context, getWorkgroupKTiledMarker()), matchDisjunction  
   StringAttr::get(context, getWorkgroupMemoryMarker())},  
  StringAttr::get(context, getVectorizeMarker()); replacement
```



线程束级分块过程实现

- (1) 计算线程束级分块大小
- (2) 获取操作并行循环范围与处理器信息
- (3) 生成线程束级分块循环体逻辑
- (4) 生成线程束级分块循环嵌套



```
"linalg.matmul"(%22, %23, %24) <{...}> ({...}) {__internal_linalg_transform__ = "workgroup_memory", ...}: ...
```

线程束级分块大小 = 设备级分块大小 / 线程束数量

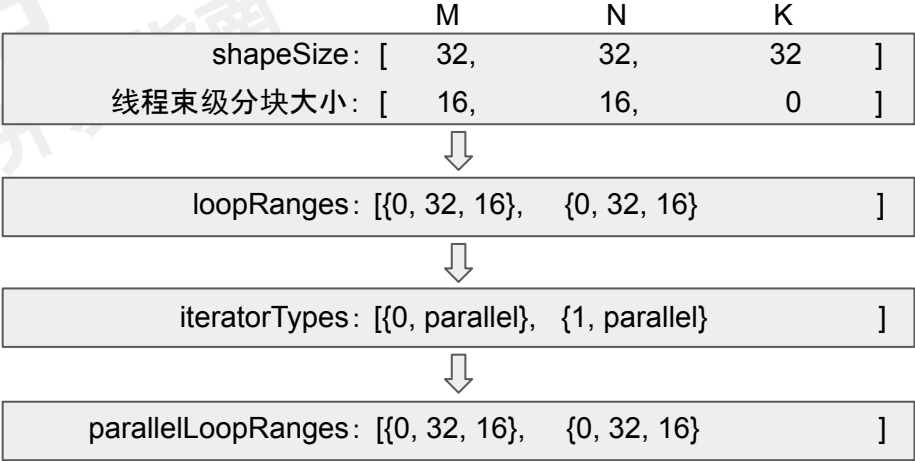
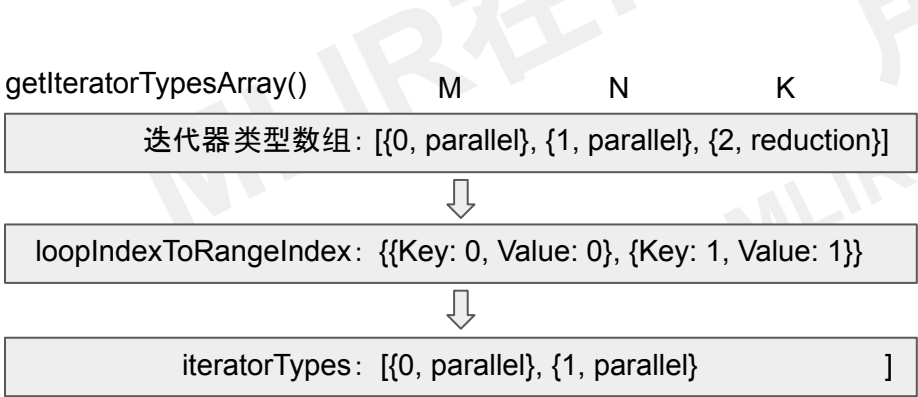
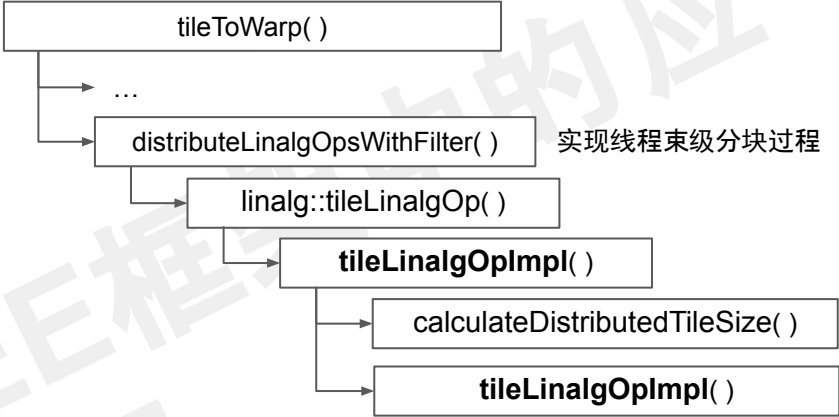
$\text{warpPerWorkgroup} = \text{workgroup_size} / \text{kWarpSize}$

```
func.func @matmul_xxx() attributes {translation_info =  
#iree_codegen.translation_info<LLVMGPUMatmulTensorCoreMmaSync  
workgroup_size = [64, 2, 1] subgroup_size = 32, ...>}
```



线程束级分块过程实现

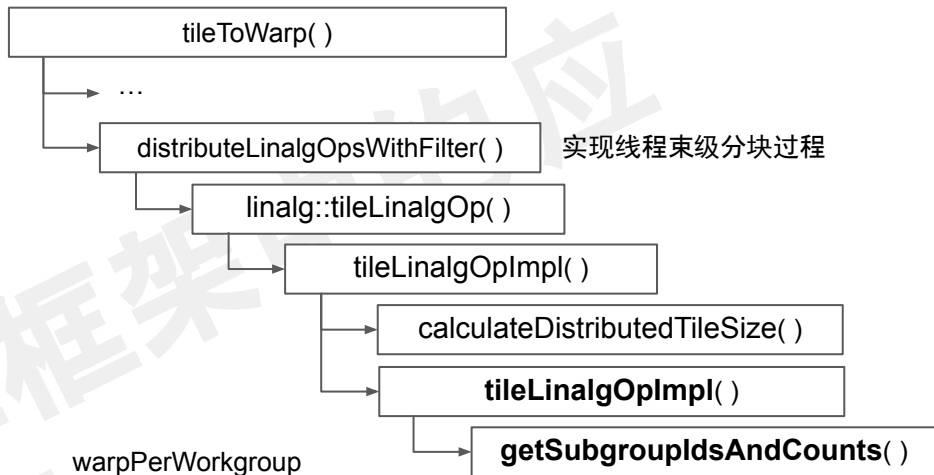
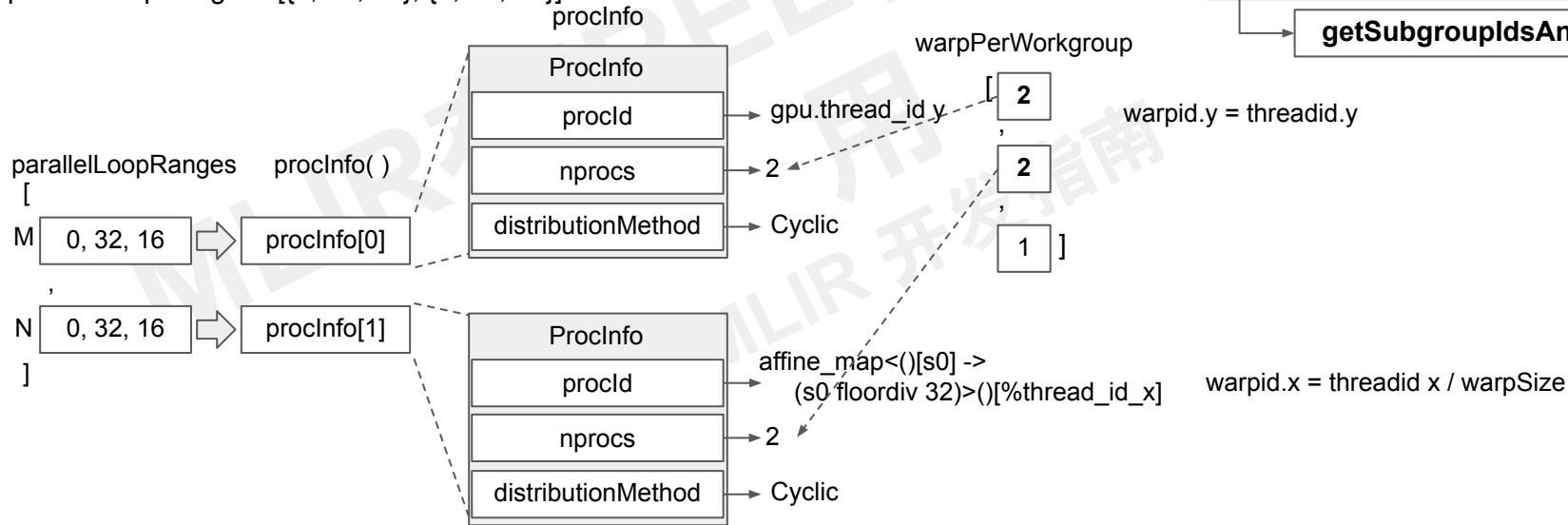
- (1) 计算线程束级分块大小
- (2) 获取操作并行循环范围与处理器信息



线程束级分块过程实现

- (1) 计算线程束级分块大小
- (2) 获取操作并行循环范围与处理器信息

parallelLoopRanges: [{0, 32, 16}, {0, 32, 16}]

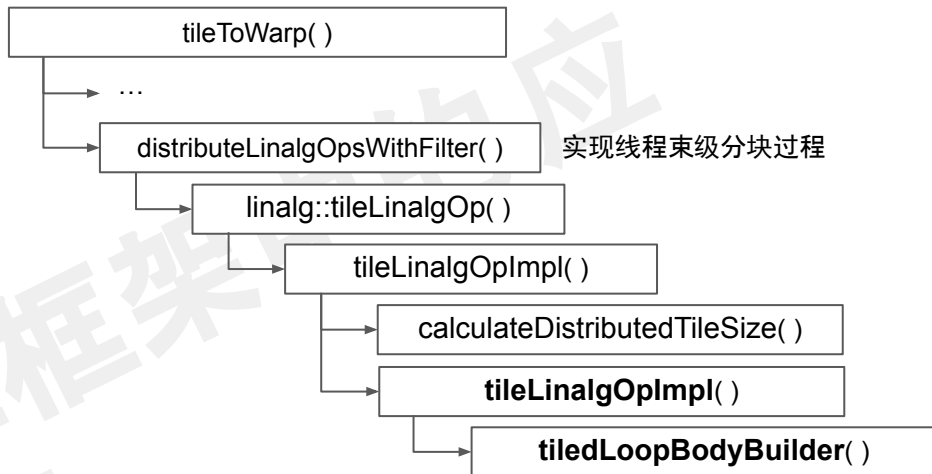


线程束级分块过程实现

- (1) 计算线程束级分块大小
- (2) 获取操作并行循环范围与处理器信息
- (3) 生成线程束级分块循环体逻辑

线程束级分块大小: [16, 16, 0]

```
%subview_8 = memref.subview %alloc_0[%arg1, 0] [16, 32] [1, 1] :  
memref<32x32xf16, ...<workgroup>> to memref<16x32xf16, ...>, ...<workgroup>>  
%subview_9 = memref.subview %alloc_0, %arg2] [32, 16] [1, 1] : memref<32x32xf16,  
...<workgroup>> to memref<32x16xf16, ...>, ...<workgroup>>  
%subview_10 = memref.subview %alloc_1[%arg1, %arg2] [16, 16] [1, 1] :  
memref<32x32xf16, ...<workgroup>> to memref<16x16xf16, ...>, ...<workgroup>>
```



线程束级分块过程实现

- (1) 计算线程束级分块大小
- (2) 获取操作并行循环范围与处理器信息
- (3) 生成线程束级分块循环体逻辑
- (4) 生成线程束级分块循环嵌套

loopRanges: [{0, 32, 16}, {0, 32, 16}]

%44 = 0, %45 = 32, %46 = 16
%47 = 0, %48 = 32, %49 = 16

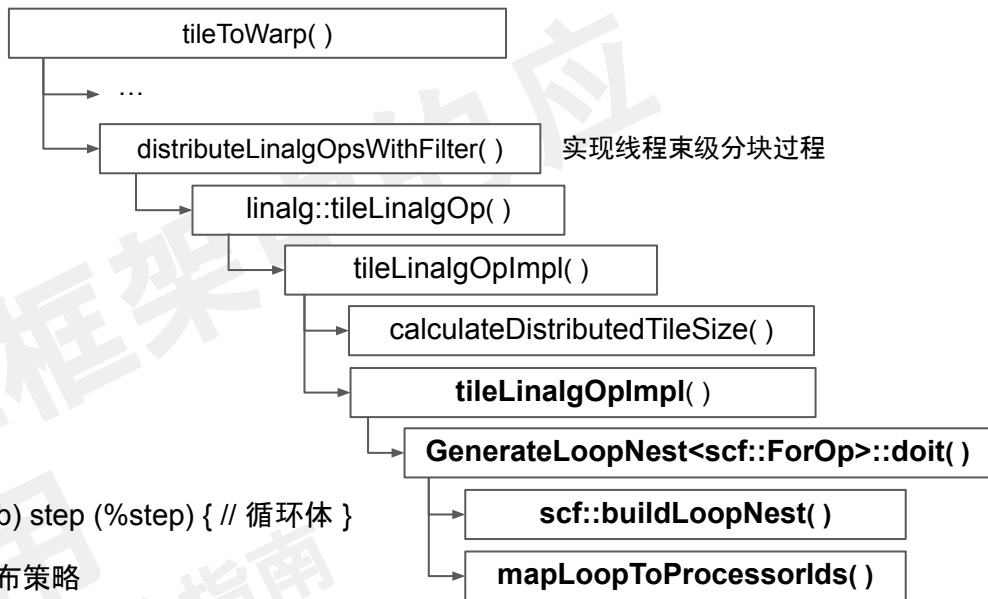
```
"scf.for"(%44, %45, %46) ({
...
"scf.for"(%47, %48, %49) ({
...
}) : (index, index, index) -> ()
...
}) : (index, index, index) -> ()
```

scf.parallel (%iv) = (%lb) to (%ub) step (%step) { // 循环体 }

↓ Cyclic 循环分布策略

scf.parallel (%iv) = (%lb + %procd * %step) to (%ub) step (%step * %nprocs) { // 循环体 }

```
%thread_id_x = gpu.thread_id x
%thread_id_y = gpu.thread_id y
%6 = affine.apply affine_map<()>[s0] -> (s0 * 16)>()[%thread_id_y]
scf.for %arg1 = %6 to %c32 step %c32 {
  %7 = affine.apply affine_map<(d0) -> ((d0 floordiv 32) * 16)>()[%thread_id_x]
  scf.for %arg2 = %7 to %c32 step %c32 {
    ...
  }
}
```



线程束级分块过程实现

```

%thread_id_x = gpu.thread_id x
%thread_id_y = gpu.thread_id y
%6 = affine.apply affine_map<() [s0] -> (s0 * 16)>()[%thread_id_y]
scf.for %arg1 = %6 to %c32 step %c32 {
  %7 = affine.apply affine_map<(d0) -> ((d0 floordiv 32) * 16)>(%thread_id_x)
  scf.for %arg2 = %7 to %c32 step %c32 {
    %subview_8 = memref.subview %alloc_0[%arg1, 0] [16, 32] [1, 1] :
memref<32x32xf16, ...> to memref<16x32xf16, ...>, ...>
    %subview_9 = memref.subview %alloc[0, %arg2] [32, 16] [1, 1] :
memref<32x32xf16, ...> to memref<32x16xf16, ...>, ...>
    %subview_10 = memref.subview %alloc_1[%arg1, %arg2] [16, 16] [1, 1] :
memref<32x32xf16, ...> to memref<16x16xf16, ...>, ...>
    linalg.matmul {...} ins(%subview_8, %subview_9 : memref<16x32xf16, ...>,
...>, memref<32x16xf16, ...>, ...>) outs(%subview_10 : memref<16x16xf16,
...>, ...>)
  }
}

```

```

%7 = affine.apply affine_map<(d0) ->
((d0 floordiv 32) * 16)>(%thread_id_x)

```

```

%6 = affine.apply affine_map<() [s0] ->
(s0 * 16)>()[%thread_id_y]

```

